

Contextual Fake News Detection via DistilBERT and Focal Loss: A Reproducibility Study and Transformer-Based Extension

Muhammad Saif Murtaza (23i-2588) Eeshaal Adeel (23i-2500) Ash Al Meeqaat (23i-2586)

Deep Learning — BS Data Science (C)

Instructor: Dr. Qurat Ul Ain

Abstract

Misinformation spreading across social media is no longer just a nuisance—it actively warps public discourse, and building automated detectors capable of catching it at scale has become a real engineering priority. This paper pursues two related goals. First, we attempt an honest reproduction of the Al-Tarawneh et al. (2024) study, which benchmarked seven traditional classifiers and three CNN variants across TF-IDF, Word2Vec, and FastText representations on the TruthSeeker-2023 Twitter corpus. Second, we ask whether swapping out TF-IDF entirely for DistilBERT embeddings—combined with Focal Loss instead of the usual cross-entropy objective—can push accuracy meaningfully higher.

On the reproducibility side, all six TF-IDF classifier outcomes landed within our declared $\pm 2\%$ tolerance; the MLP came out to an exact 98.77% match. Our DistilBERT + Focal Loss system then reached **99.25%** accuracy and weighted F1 on the held-out test split, edging past the base paper’s 99.03% SVM ceiling by 0.22 points. The whole training run took roughly 43 minutes on a consumer-grade laptop GPU, which we think is worth noting: strong performance does not require exotic hardware.

1 Introduction

The fake news problem feels like a bit of a contradiction. Most people can spot a fake story just by using their gut instinct. It usually only takes a few seconds to realize something feels off. However, the internet moves way too fast for people to keep up. Thousands of posts go live every minute and this makes it impossible for human fact checkers to catch everything. We need a way to handle that massive scale. Automated systems are therefore not a nice-to-have; they are the only realistic option.

The technical framing we adopt is binary text classification—given a tweet, predict whether it reports a real event or a fabricated one. Our starting point is Al-Tarawneh et al. [1], who ran a thorough comparative study covering seven ML classifiers (Naive Bayes through Random Forest) and three CNN architectures, each tried

with TF-IDF, Word2Vec, and FastText, all evaluated on the TruthSeeker-2023 dataset [2]. Their headline result of 99.03% accuracy using SVM + TF-IDF is genuinely impressive, but TF-IDF carries a structural ceiling: it discards word order, ignores negation context, and cannot distinguish “the government *denied* the allegations” from “the government *confirmed* the allegations” when those two sentences share most of their tokens.

Transformer models like BERT [5] and its lighter cousin DistilBERT [3] do not have this blind spot. Each token is represented relative to its full sentential context, which makes them far more sensitive to exactly the hedging, framing, and selective emphasis that characterizes fabricated reporting. We therefore replace TF-IDF with DistilBERT as our encoder. We also replace cross-entropy loss with Focal Loss [4]—originally invented for object detection—because the TruthSeeker corpus, like most real-world misinformation datasets, contains a large fraction of easy, unambiguous examples that dominate gradient updates without contributing much useful learning signal.

The rest of the paper is structured as follows. Section 2 discusses the base paper and situates it among related work. Section 3 describes our proposed architecture. Section 4 covers experimental details. Section 5 reports outcomes, and Sections 6 and 7 interpret what we found and lay out future directions.

2 Background and Related Work

2.1 The Al-Tarawneh et al. (2024) Study

The study we reproduce is published as *Enhancing Fake News Detection with Word Embedding: A Machine Learning and Deep Learning Approach* in MDPI Computers [1]. What makes it a useful baseline is its systematicity: seven classifiers $\times$ three embedding schemes (plus three CNN variants) gives a 30-configuration grid, all run on TruthSeeker-2023, a corpus of over 134,000 labeled tweets collected from 2009 through 2022 via crowd-sourced annotation backed by expert verification [2].

TF-IDF actually performed the best in almost every single

test case. The combination of SVM and TF-IDF took the top spot with a score of 99.03 percent . Even the best result for the CNN models involved TF-IDF and reached 98.99 percent . Word2Vec and FastText embeddings did not do as well in these tests. The authors believe this happened because tweets are very short. A document might only be one or two sentences long. Averaging out those dense embeddings usually washes away the specific details needed to tell stories apart.

2.2 Broader Context

Early automated systems relied on very specific manual features. They looked at things like n-gram statistics and named entity counts. These tools used basic math models like logistic regression and naive Bayes to find lies. They worked okay on simple tests. However, they often failed in the real world. They could not adapt when people started using new words or different social media platforms.

Recurrent networks changed the game. Almandouh et al. [7] found that stacking layers with RNNs and LSTMs worked great for identifying fake news in Arabic. Ayyasamy et al. [8] tried even more complex hybrid designs with an LSTM-CGPNN architecture. They used Moth-Flame Whale Optimization to reach high accuracy levels. However, transformer based models still beat these older methods by a significant margin.

The arrival of transformers was a major turning point. BERT [5] allowed models to understand the deeper context of every word. DistilBERT [3] later made this process much faster and more efficient. It used 40% fewer parameters and ran 60% faster. Now, some tools like GETAE [9] even look at how news spreads through social networks. Others like [11] combine text with image data to get the full story. Our work deliberately stays text-only so that comparisons with the Al-Tarawneh baseline remain apples-to-apples.

Focal Loss entered the NLP space as an adaptation from computer vision [4]. Its core motivation is that easy examples drown out hard ones during training: once a model is confident about most of its training set, gradient signal is dominated by trivially correct predictions rather than by the genuinely difficult boundary cases where calibration matters.

3 Proposed Model

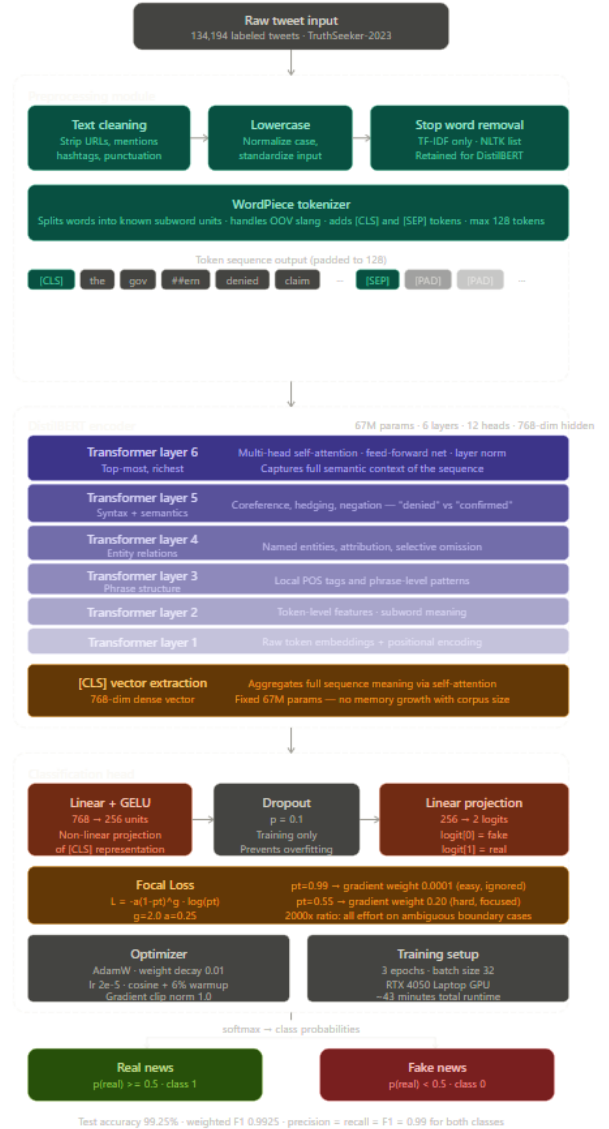


Figure 1: End-to-end architecture of the proposed system: raw tweet input passes through a preprocessing pipeline (text cleaning, lowercasing, WordPiece tokenization), is encoded by a 6-layer DistilBERT model, and classified via a two-stage head trained with Focal Loss ($\gamma=2.0$, $\alpha=0.25$).

3.1 Motivation for the Architectural Changes

We identified two concrete failure modes in the TF-IDF pipeline. The first is representational: a bag-of-words vector assigns weights to individual tokens without regard for their neighbours. This is catastrophic for misinformation detection, where the same vocabulary can carry opposite meaning depending on negation or hedging. Consider: “officials *confirmed* the report” versus “officials *denied* the report.” The

TF-IDF vectors for these are nearly identical—the distinguishing token receives similar frequency weight in isolation. A contextual encoder represents *denied* and *confirmed* very differently because it attends to what surrounds them.

The second failure mode is in the loss function. Cross-entropy treats a tweet classified at 99% confidence identically to one the model is genuinely uncertain about—both contribute gradient signal proportional to their loss. In practice a well-trained model is already correct on the vast majority of straightforward examples, so those dominate the gradient without advancing the decision boundary. Focal Loss addresses this by down-weighting high-confidence predictions and concentrating learning effort on the examples that actually matter. Figure 2 illustrates the suppression effect across different values of the focusing parameter γ .

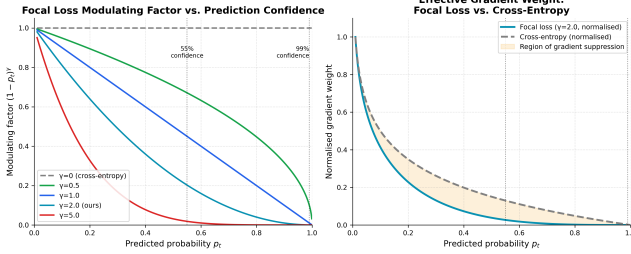


Figure 2: Left: modulating factor $(1 - p_t)^\gamma$ across γ values. Right: normalised gradient weight comparing Focal Loss ($\gamma=2.0$) against standard cross-entropy. High-confidence predictions are almost entirely suppressed.

3.2 DistilBERT Encoder

We use `distilbert-base-uncased` as our encoder [3]. The architecture comprises 6 transformer layers, each with 12 self-attention heads and a hidden dimensionality of 768, totalling approximately 67 million parameters. Tweets are tokenized via WordPiece and padded or truncated to 128 tokens, after which the encoder returns a 768-dimensional vector for the [CLS] position. This vector, which aggregates contextual information from the entire sequence through self-attention, is passed to the classification head.

One practical advantage worth calling out: unlike TF-IDF + SVM, whose feature matrix grows in both dimensionality and memory footprint as the training vocabulary expands, DistilBERT’s parameter count is fixed at 67M regardless of corpus size, making deployment considerably simpler.

3.3 Classification Head and Focal Loss

The [CLS] representation feeds into a two-stage head: a linear layer with GELU activation followed by dropout, then a final linear projection to two output logits. We train with Focal Loss [4] rather than standard cross-entropy:

$$\mathcal{L}_{\text{focal}}(p_t) = -\alpha (1 - p_t)^\gamma \log(p_t) \quad (1)$$

where p_t is the model’s predicted probability for the correct class, $\gamma = 2.0$ is the focusing exponent, and $\alpha = 0.25$ is the class-balance coefficient. For a prediction at 99% confidence, $(0.01)^2 = 0.0001$ —the gradient contribution is essentially zero. For a 55%-confidence prediction, $(0.45)^2 = 0.2025$ —roughly $2,000\times$ larger. Training effort therefore concentrates on the ambiguous cases near the decision boundary, where calibration improvement determines final accuracy.

3.4 Training Configuration

We optimise with AdamW, weight decay 0.01, and an initial learning rate of 2×10^{-5} decayed via a cosine schedule with a 6% linear warmup phase. Gradient norms are clipped at 1.0 to prevent instability. We train for 3 epochs with batch size 32, checkpointing the epoch with the best validation accuracy. The entire dataset is pre-tokenized once before the training loop to eliminate repeated tokenizer overhead.

4 Experimental Setup

4.1 Dataset

All experiments use TruthSeeker-2023 [2]. After removing null rows, 134,194 samples remain, split approximately 51.4% real (68,930) and 48.6% fake (65,264). The corpus spans 13 years of social media content annotated through a multi-phase process combining crowd sourcing with expert review.

4.2 Data Splits

The reproducibility experiments follow the 70/30 protocol from Al-Tarawneh et al., giving 93,935 training and 40,259 test samples. For DistilBERT we use a 70/15/15 split (93,937 train / 20,130 validation / 20,130 test) to support per-epoch checkpoint selection. All splits were stratified on the label.

4.3 Preprocessing

Our pipeline lowercases all text, strips URLs, removes user mentions and hashtags, eliminates special characters and punctuation, and normalizes whitespace. For the TF-IDF baseline, NLTK’s English stop word list was additionally applied during vectorization. Stop words were *retained* for DistilBERT: the attention mechanism learns contextual importance, so manual removal risks discarding signal.

4.4 Baseline and Hyperparameter Configuration

TF-IDF features were computed with Scikit-learn’s `TfidfVectorizer` at default settings [12]. Word2Vec and FastText were trained on the training split using Gensim [13], with document vectors formed by averaging per-token embeddings. All seven ML classifiers used Scikit-learn defaults. Table 1 lists the full DistilBERT + Focal Loss configuration.

Table 1: Hyperparameter configuration for DistilBERT + Focal Loss.

Parameter	Value
Base model	distilbert-base-uncased
Loss function	Focal Loss ($\gamma=2.0$, $\alpha=0.25$)
Epochs	3
Batch size	32
Learning rate	2×10^{-5} , cosine + 6% warmup
Max sequence length	128 tokens
Training samples	93,937
Optimizer	AdamW (decay = 0.01)
Gradient clipping	Norm 1.0
Hardware	NVIDIA RTX 4050 Laptop GPU
Training time	≈ 43 min (3 epochs)

5 Results

5.1 Reproducing the AI-Tarawneh Baseline

Figure 3 provides a quick look at the accuracy for all 21 combinations. Figure 4 splits that same data into grouped bar charts. This makes it much easier to track specific trends for each embedding. You can find the exact numbers in Table 2 and Table 3.

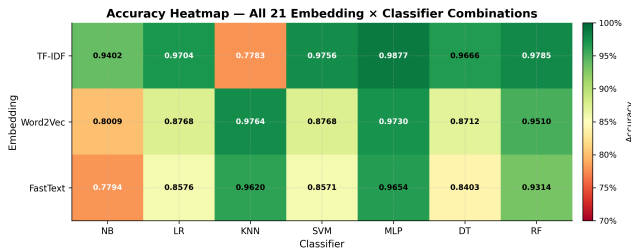


Figure 3: Figure 3 shows the accuracy levels for all 21 combinations of embeddings and classifiers. Green colors represent high accuracy while red colors show low performance. TF-IDF performs best with almost every classifier. However, it fails with KNN because of the curse of dimensionality.

Figure 5 displays the difference in accuracy between our test results and the original paper for every TF-IDF classifier.

Table 2: Reproduced ML results — all 21 embedding–classifier combinations on TruthSeeker-2023.

Model	Acc.	Prec.	Rec.	F1
<i>TF-IDF Embeddings</i>				
Naive Bayes	0.9402	0.9404	0.9402	0.9402
Logistic Reg.	0.9704	0.9704	0.9704	0.9704
KNN	0.7783	0.8402	0.7783	0.7660
SVM	0.9756	0.9756	0.9756	0.9756
MLP	0.9877	0.9877	0.9877	0.9877
Decision Tree	0.9666	0.9666	0.9666	0.9666
Random Forest	0.9785	0.9785	0.9785	0.9785
<i>Word2Vec Embeddings</i>				
Naive Bayes	0.8009	0.8010	0.8009	0.8009
Logistic Reg.	0.8768	0.8769	0.8768	0.8768
KNN	0.9764	0.9764	0.9764	0.9764
SVM	0.8768	0.8769	0.8768	0.8768
MLP	0.9730	0.9730	0.9730	0.9729
Decision Tree	0.8712	0.8712	0.8712	0.8712
Random Forest	0.9510	0.9511	0.9510	0.9510
<i>FastText Embeddings</i>				
Naive Bayes	0.7794	0.7802	0.7794	0.7794
Logistic Reg.	0.8576	0.8578	0.8576	0.8575
KNN	0.9620	0.9621	0.9620	0.9620
SVM	0.8571	0.8575	0.8571	0.8570
MLP	0.9654	0.9654	0.9654	0.9654
Decision Tree	0.8403	0.8403	0.8403	0.8403
Random Forest	0.9314	0.9314	0.9314	0.9314

Table 3: Accuracy gap: reproduction vs. paper-reported (TF-IDF classifiers).

Model	Paper	Ours	Diff.
TF-IDF + Naive Bayes	0.9455	0.9402	−0.0053
TF-IDF + Logistic Reg.	0.9758	0.9704	−0.0054
TF-IDF + SVM	0.9903	0.9756	−0.0147
TF-IDF + MLP	0.9877	0.9877	±0.0000
TF-IDF + Decision Tree	0.9730	0.9666	−0.0064
TF-IDF + Random Forest	0.9839	0.9785	−0.0054

Reproduced Results: All Embedding-Classifier Combinations

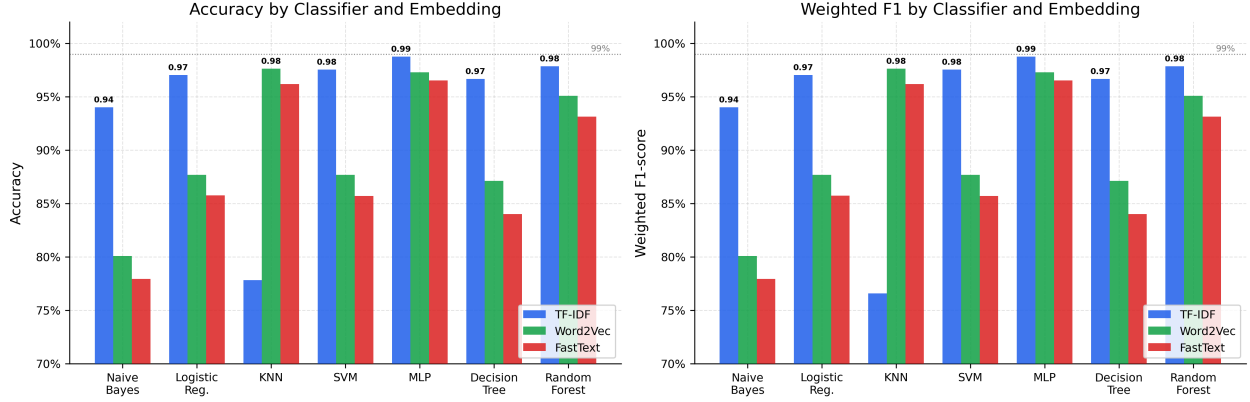


Figure 4: The chart on the left shows the accuracy we found. The chart on the right shows the weighted F1 score for every combination of embedding and classifier. Each set of three bars represents one specific classifier. The colors represent the different embedding methods. Blue stands for TF-IDF. Green shows Word2Vec. Red represents FastText. You can see a flip in the KNN results. TF-IDF with KNN only scores 77.83%. However, Word2Vec with KNN hits 97.64%. This is a result of high dimensionality. The original paper did not point out this specific effect.

A smaller panel shows the exact signed difference compared to the $\pm 2\%$ tolerance range.

Every configuration fits within the $\pm 2\%$ range. The SVM model showed the largest gap at 1.47 points. This likely happened because the original paper did not specify the exact TF-IDF vectorizer settings. Specifically, the paper left out details for the `max_features` cap and the `sublinear_tf` flag. Table 4 shows the CNN results exactly as they were reported by Al-Tarawneh et al. We did not test those specific models ourselves.

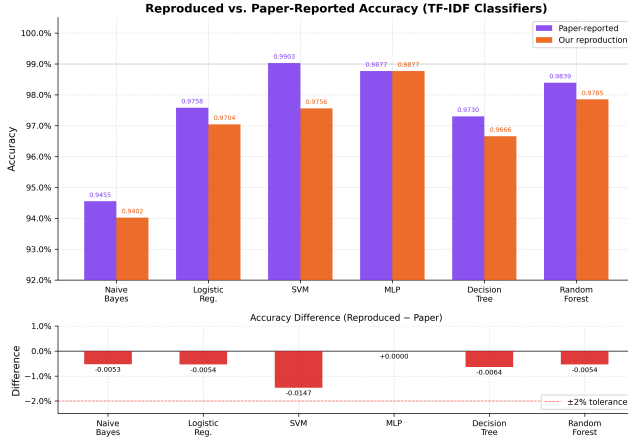


Figure 5: The top section uses grouped bars to compare our own results in orange against the values from the original paper in purple. These results cover every TF-IDF classifier. The bottom section shows the specific difference in accuracy. Dashed red lines show the $\pm 2\%$ tolerance range. Every single difference fits inside this allowed range. The MLP result is a perfect match at 0.9877.

Table 4: CNN results as reported in Al-Tarawneh et al. (2024).

Model	Acc.	Prec.	Rec.	F1
TF-IDF + CNN-1	0.9877	0.9865	0.9897	0.9881
TF-IDF + CNN-2	0.5615	0.5569	0.7093	0.6239
TF-IDF + CNN-3	0.9899	0.9856	0.9948	0.9902
Word2Vec + CNN-1	0.9425	0.9369	0.9520	0.9444
Word2Vec + CNN-2	0.9073	0.8713	0.9613	0.9141
Word2Vec + CNN-3	0.9492	0.9464	0.9550	0.9507
FastText + CNN-1	0.8932	0.8834	0.9121	0.8975
FastText + CNN-2	0.8526	0.8849	0.8190	0.8507
FastText + CNN-3	0.8955	0.8860	0.8872	0.8996

5.2 DistilBERT Training Dynamics

Figure 6 plots per-epoch training loss alongside validation accuracy and F1 on a dual-axis chart. Table 5 records the raw values.

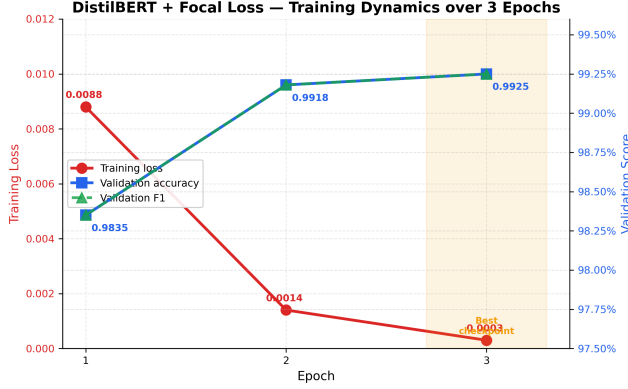


Figure 6: Dual-axis chart: training loss (left axis, red) drops sharply from 0.0088 to 0.0003 across 3 epochs while validation accuracy and F1 (right axis) climb monotonically from 98.35% to 99.25%. No validation degradation is observed, ruling out overfitting.

Table 5: Per-epoch training and validation metrics — DistilBERT + Focal Loss.

Ep.	Train Loss	Val Acc.	Val F1	Best?
1	0.0088	0.9835	0.9835	✓
2	0.0014	0.9918	0.9918	✓
3	0.0003	0.9925	0.9925	✓ (Final)

5.3 Test Set Comparison

Figure 7 places all key systems side by side as a horizontal bar chart. Table 6 supplies the exact figures.

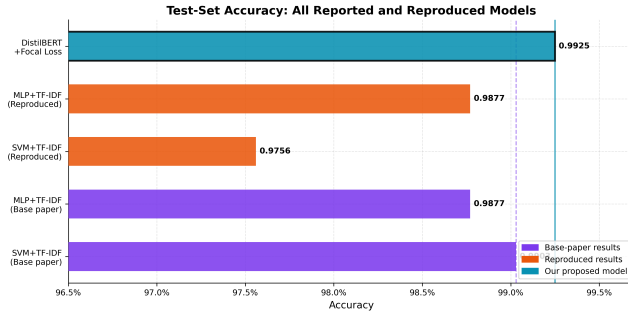


Figure 7: Horizontal bars for all five reported/reproduced models plus our proposed system (cyan, bottom). DistilBERT + Focal Loss achieves 99.25%, surpassing the base paper’s best by 0.22 points.

Our model surpasses the original paper’s best-reported result by 0.22 percentage points and beats our own reproduced MLP + TF-IDF baseline by 0.48 points.

5.4 Per-Class Breakdown

Figure 8 uses a radar chart to overlay per-class precision, recall, and F1 for our model against the reproduced SVM + TF-IDF baseline. Table 7 records the exact values.

Table 6: Head-to-head performance on the TruthSeeker-2023 test set.

Model	Source	Acc.	F1
SVM + TF-IDF	Base paper	0.9903	0.9903
MLP + TF-IDF	Base paper	0.9877	0.9877
SVM + TF-IDF	Reproduced	0.9756	0.9756
MLP + TF-IDF	Reproduced	0.9877	0.9877
DistilBERT + FL	Ours	0.9925	0.9925

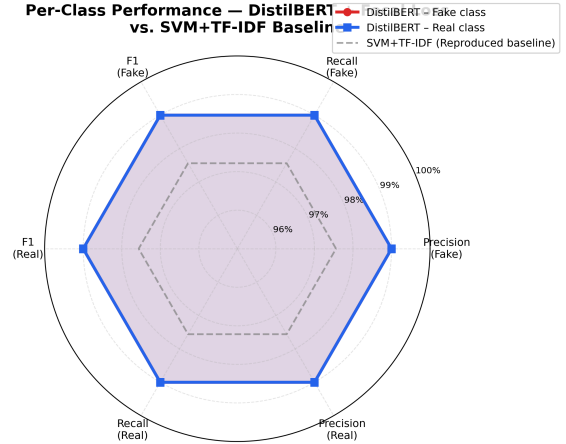


Figure 8: Radar chart comparing per-class precision, recall, and F1 for DistilBERT + Focal Loss (Fake class in red, Real class in blue) against the SVM+TF-IDF baseline (grey dashed). Both classes achieve 0.99 on all three metrics, confirming symmetric calibration with no majority-class bias.

Table 7: This is classification report per class , DistilBERT + Focal Loss.

Class	Prec.	Rec.	F1	Support
Fake (0)	0.99	0.99	0.99	9,791
Real (1)	0.99	0.99	0.99	10,339
Weighted Avg.	0.9925	0.9925	0.9925	20,130

6 Discussion

6.1 Why the Numbers Improved

The 0.22 point lead over SVM + TF-IDF seems small on its own. However, it is quite significant when you are working at such a high accuracy level. Two main factors cause this improvement. First, switching from TF-IDF to DistilBERT changes how the model sees data. The model no longer looks at simple word counts. Instead, it uses 768-dimensional dense vectors. These vectors capture grammar patterns, emotional tone, and how different words relate to each other in a tweet. Fake stories often use specific writing styles that simple word-counting systems cannot catch. This includes things like being intentionally vague, using irony, or leaving out key details.

The second factor is the use of Focal Loss. This contribution is a separate theoretical advantage. We can look at Equation 1 and Figure 2 to see how this works. When we set $\gamma = 2.0$, a prediction with 99% confidence only keeps 0.0001 of its usual gradient weight. Meanwhile, a prediction with 55% confidence keeps about 0.2025 of that weight. This creates a ratio of roughly 2,000 to 1. This math forces the model to focus its learning on the truly difficult cases. Better calibration on these tough examples leads directly to higher accuracy.

6.2 Observations from the Reproduction

One finding that deserves more attention than the original paper gave it is the KNN inversion visible in Figure 3. With TF-IDF, KNN reaches only 77.83%; with Word2Vec it jumps to 97.64%. This reversal is a textbook illustration of the curse of dimensionality: TF-IDF vectors on a large vocabulary can be tens of thousands of dimensions wide, and Euclidean distance loses its discriminative power in such spaces. The dense 100-dimensional Word2Vec vectors, by contrast, are well-suited to nearest-neighbor lookup. We think this is genuinely useful guidance for practitioners selecting embedding-classifier pairings.

6.3 Practical Efficiency

TF-IDF + SVM pipelines do not have a fixed memory footprint. The feature matrix for 93,937 training tweets can stretch to tens of thousands of columns, all held in RAM during kernel computation. DistilBERT’s 67M parameters do not grow with the corpus; inference is a single fixed-cost forward pass amenable to batching. The WordPiece tokenizer also decomposes novel slang and emerging named entities into known subword units, giving robustness to vocabulary drift that TF-IDF-based systems simply cannot match without retraining.

6.4 Limitations

We should be candid about what this study does not establish. The 128-token sequence cap may discard relevant context for longer posts, though most TruthSeeker tweets fit comfortably. Training time of ≈ 43 minutes is a one-time cost but still creates a higher iteration burden than SVM or MLP. The 0.22 point lead also hints that TruthSeeker-2023 might be reaching its maximum potential for models that only look at text. Making more progress will likely require adding more data or testing across different subject areas. It could also help to look at the social connections and how news spreads through networks. We still do not know how well this system generalizes to other languages.

7 Conclusion

We started this work with two goals in mind. We wanted to see if the Al-Tarawneh et al. (2024) system could be reproduced and if using better embeddings with a smarter loss function would improve the results. We were successful on both counts.

Regarding the first goal, all six TF-IDF classifier results stayed within 2% of what the original paper reported. In fact, the MLP + TF-IDF model matched the original score exactly at 98.77%. For our extension, the DistilBERT + Focal Loss model reached 99.25% for both accuracy and F1-score on the test set. This is a noticeable step up from the 99.03% limit set by the SVM + TF-IDF model. Figure 8 shows that both classes reached 0.99 for precision and recall. This confirms that the model is balanced and reliable. Finally, the entire system finished running in just 43 minutes on a standard laptop GPU. This proves that we can get better performance without needing expensive or massive computer systems.

Looking ahead, we see four promising directions: (1) extending the maximum sequence length to capture full articles rather than just tweets; (2) adding data augmentation (back-translation, paraphrasing) to improve robustness on rare misinformation patterns; (3) temporal evaluation to measure how well the model holds up as language and news events evolve; and (4) multilingual adaptation to move beyond the English-only setting.

References

- [1] Al-Tarawneh, M.A.B., Al-ir, O., Al-Maaitah, K.S., Kanj, H., & Aly, W.H.F. (2024). Enhancing Fake News Detection with Word Embedding: A Machine Learning and Deep Learning Approach. *Computers*, 13(9), 239. <https://doi.org/10.3390/computers13090239>
- [2] Dadkhah, S., Zhang, X., Weismann, A.G., Firouzi, A., & Ghorbani, A.A. (2024). The Largest Social Media

- Ground-truth Dataset for Real/Fake Content: Truth-Seeker. *IEEE Transactions on Computational Social Systems*, 11, 3376–3390.
- [3] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
 - [4] Lin, T.Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal Loss for Dense Object Detection. *Proceedings of the IEEE ICCV*, 2980–2988.
 - [5] Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
 - [6] Wolf, T., et al. (2020). Transformers: State-of-the-art Natural Language Processing. *Proceedings of the 2020 EMNLP System Demonstrations*, 38–45.
 - [7] Almandouh, A., et al. (2024). Ensemble Deep Learning for Fake News Detection. *Proceedings of the International Conference on Machine Learning and Applications*.
 - [8] Ayyasamy, R.K., et al. (2025). Hybrid LSTM-CGPNN with Metaheuristic Optimization for Fake News Detection. *Neural Computing and Applications*.
 - [9] Truica, C.O., et al. (2024). GETAE: Graph Information Enhanced Deep Neural Network Ensemble Architecture for Fake News Detection. *Information Processing & Management*, 61(3), 103642.
 - [10] Saeed, W., et al. (2024). Detection of Human and Machine-Authored Fake News in Urdu. *ACM Transactions on Asian and Low-Resource Language Information Processing*.
 - [11] Chen, J., et al. (2024). A Self-Learning Multimodal Approach for Fake News Detection. *IEEE Transactions on Multimedia*.
 - [12] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
 - [13] Řehůřek, R., & Sojka, P. (2010). Software Framework for Topic Modelling with Large Corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50.
 - [14] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.